

Aula 08/15

Escola Tecnológica FPFtech

Curso Técnico: Informática para Internet

Módulo: Testes de Software

Prof. Denis Moraes Guimarães

01 de junho de 2026

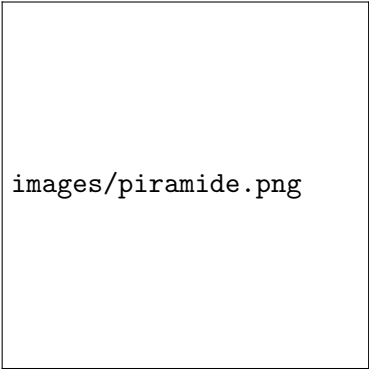
Sumário

- ▶ Exemplo
- ▶ Estrutura
- ▶ Pré-condições
- ▶ Pós-condições
- ▶ Boas práticas
- ▶ Mock e Stub
- ▶ Cobertura
- ▶ Escolha de projeto

Exemplo

Pirâmide de testes e níveis - Base

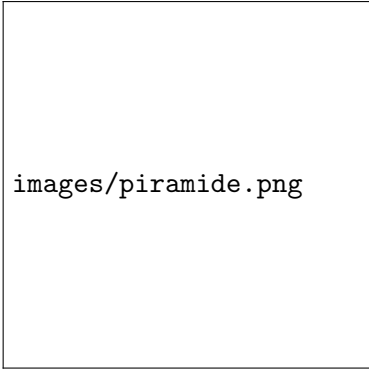
- ▶ Base da pirâmide (muitos testes).
- ▶ Testes de unidade.
- ▶ São os mais rápidos, simples e baratos.
- ▶ Testam pequenas partes do código isoladamente (funções, métodos).
- ▶ Devem ser a maioria dos testes.



images/piramide.png

Pirâmide de testes e níveis - Meio

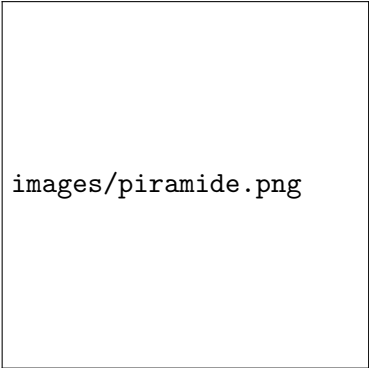
- ▶ Meio da pirâmide (quantidade moderada)
- ▶ Testes de integração.
- ▶ Verificam a comunicação entre módulos (ex: banco de dados + API).
- ▶ São mais lentos e complexos que os de unidade.
- ▶ Existem em menor quantidade.

A diagram of a pyramid of tests, divided into three horizontal sections. The middle section is highlighted with a light blue background. The text 'images/piramide.png' is centered within the middle section.

images/piramide.png

Pirâmide de testes e níveis - Topo

- ▶ Testes de sistema / ponta a ponta (E2E)
- ▶ Simulam o uso real do sistema do início ao fim.
- ▶ São os mais lentos, frágeis e caros de manter.
- ▶ Devem ser poucos.



images/piramide.png

Testes funcionais

- ▶ Teste de unidade
- ▶ Teste de integração
- ▶ Teste de sistema
- ▶ Teste de ponta a ponta (E2E)
- ▶ Teste de aceitação (UAT)
- ▶ Teste de regressão
- ▶ Teste de sanidade
- ▶ Teste de fumaça
- ▶ Teste de repetibilidade

Testes funcionais - Unidade

```
1 import unittest
2
3 class TestStringMethods(unittest.TestCase):
4
5     def test_upper(self):
6         self.assertEqual('foo'.upper(), 'FOO')
7
8     def test_isupper(self):
9         self.assertTrue('FOO'.isupper())
10        self.assertFalse('Foo'.isupper())
11
12    def test_split(self):
13        s = 'hello world'
14        self.assertEqual(s.split(), ['hello', 'world'])
15        with self.assertRaises(TypeError):
16            s.split(2)
17
18 if __name__ == '__main__':
19     unittest.main()
```


Testes funcionais - Integração

```
1 import unittest
2 import requests
3
4 class TestHTTTPublishAndResponse(unittest.TestCase):
5     BASE_URL = "http://localhost:5000"
6     def test_publish_and_get_response(self):
7         resp = requests.post(
8             f"{self.BASE_URL}/publish",
9             json={"message": "ping"}
10        )
11
12        self.assertEqual(resp.status_code, 200)
13        resp = requests.get(f"{self.BASE_URL}/response")
14        self.assertEqual(resp.status_code, 200)
15        data = resp.json()
16        self.assertIn("message", data)
17        self.assertEqual(data["message"], "pong")
18
19 if __name__ == "__main__":
20     unittest.main()
```

Testes funcionais - Sistema

Ambiente o mais próximo possível da produção

`images/sistema.png`

Testes funcionais - Ponta a ponta (E2E)

Login → compra → pagamento → confirmação

`images/sistema.png`

Testes funcionais - Aceitação

Validar se o sistema atende aos requisitos do cliente

`images/sistema.png`

Testes funcionais - Regressão

"Se nova feature for adicionada, o que já existe continua funcionando?"

images/ci.png

Testes funcionais - Sanidade

Menos profundo que o regressão, "a parte principal ainda funciona?"

images/sistema.png

Testes funcionais - Fumaça

Checa se o sistema “sobe” e funciona minimamente.

`images/update.png`

Testes funcionais - Repetibilidade

"A cada X vezes, o comportamento se mantém Y vezes",
comportamento inconstante.

`images/repetibilidade.jpg`

Testes não funcionais

- ▶ Teste de performance
- ▶ Teste de segurança
- ▶ Teste de usabilidade
- ▶ Teste de compatibilidade

Testes não funcionais - Performance

- ▶ Avalia o desempenho geral do sistema.
- ▶ Tempo de resposta.
- ▶ Uso de recursos.
- ▶ Estabilidade sob carga, grandes quantidades.

images/performance.jpg

Testes não funcionais - Performance

Tipos de teste de performance:

Teste	Objetivo	Exemplo
Carga	Avaliar carga esperada	5.000 usuários simultâneos
Estresse	Encontrar limite do sistema	Aumentar carga até falhar
Escalabilidade	Verificar crescimento do sistema	Adicionar servidores
Volume	Avaliar muitos dados	Banco com milhões de registros
Capacidade	Descobrir capacidade máxima	Máximo de usuários suportados
Estabilidade	Verificar funcionamento contínuo	Sistema ativo por dias
Concorrência	Avaliar acessos simultâneos	Edição simultânea de registros
Endurance	Detectar degradação temporal	Carga constante por 48h
Pico	Avaliar aumento repentino	Explosão de acessos

Métricas:

Métrica	Descrição
Tempo de resposta	Tempo para responder uma requisição
Throughput	Quantidade de operações por segundo
Latência	Tempo entre solicitação e resposta
Taxa de erro	Percentual de falhas
Uso de CPU	Consumo do processador
Uso de memória	Consumo de RAM
Concorrência	Número de usuários simultâneos

Testes não funcionais - Segurança

- ▶ Avalia vulnerabilidades e proteção do sistema.
- ▶ Autenticação, autorização, SQL injection, ataques comuns (OWASP).



`images/seguranca.jpg`

Testes não funcionais - Usabilidade

- ▶ Verifica se o sistema é fácil de usar.
- ▶ Interface intuitiva.
- ▶ Facilidade de navegação

`images/usabilidade.jpeg`

Testes não funcionais - Compatibilidade

- ▶ Verifica funcionamento em diferentes ambientes.
- ▶ Navegadores (Chrome, Firefox)
- ▶ Sistemas operacionais.
- ▶ Dispositivos

`images/sistema.jpeg`